FlagEasy

API Helpline — every member function, explained in two lines.

FlagEasy (`flageasy::`) is a C++17 command-line flag parsing library. Workflow: **define flags** on a `Parser` → **call** `parse(argc, argv)` → **read values** from the returned `ParseResult`. Header: `flageasy/flageasy.hpp`; link against `-lflageasy`.

```
flageasy::Parser cli("myapp", "Does useful things."); cli.add_int("count", 'c', "Repeat count").default_value(1).range(1, 100); auto r = cli.parse(argc, argv); // never throws on bad user input if (r) { int n = r.as<int>("count"); }
```

Three public classes work together: **Parser** defines and runs the parse, **Flag** configures one flag fluently, **ParseResult** carries the outcome. Only definition-time programming errors (duplicate names, invalid identifiers) throw `std::invalid_argument` — everything else comes back inside the result.

1 · Parser — define flags, configure, parse

Construction

`Parser(std::string program_name, std::string description = {})`

• Creates the parser; name appears in help/usage output (paths are reduced to a basename).
• The description is printed at the top of help().

```
flageasy::Parser cli("myapp", "Does useful things.");
```

Copy is disabled; moving is allowed. Call `add_*` before parsing — flags added after parse() calls are not seen by earlier results.

Defining flags (each returns a Flag& to configure fluently)

`Flag& Parser::add_switch(std::string name, char short_name = '\0', std::string help = {})`

• Defines a boolean switch whose presence means true (--verbose).
• Supports negation (--verbose=false) and clustering (-vq); '\0' means no short form.

```
cli.add_switch("verbose", 'v', "Enable verbose output");
```

`Flag& Parser::add_string(std::string name, char short_name = '\0', std::string help = {})`

• Defines an option that takes one text value.
• All conventional syntaxes work: --name v, --name=v, -n v, -nvalue, -n=v.

```
cli.add_string("name", 'n', "Your name").required();
```

`Flag& Parser::add_int(std::string name, char short_name = '\0', std::string help = {})`

• Defines an option parsed as a signed 64-bit integer.
• Pair with .range(low, high) and .default_value(n) for validation.

```
cli.add_int("count", 'c', "Repeat count").range(1, 100);
```

`Flag& Parser::add_double(std::string name, char short_name = '\0', std::string help = {})`

• Defines an option parsed as a double-precision float.
• Accepts 2, 2.5, -0.5, 1e3; validate with .range().

```
cli.add_double("scale", 's', "Scale factor").range(0.0, 10.0);
```

`Flag& Parser::add_string_list(std::string name, char short_name = '\0', std::string help = {})`

• Defines a repeatable string option; every occurrence appends one value.
• Retrieve all values with r.as<std::vector<std::string>>(name).

```
cli.add_string_list("tag", 't', "Tags"); // -t a -t b
```

`Flag& Parser::add_int_list(std::string name, char short_name = '\0', std::string help = {})`
• Repeatable int option — each occurrence contributes one integer.
• Retrieve with r.as<std::vector<int>>(name).

```
cli.add_int_list("id");
```

`Flag& Parser::add_double_list(std::string name, char short_name = '\0', std::string help = {})`
• Repeatable double option — each occurrence contributes one double.
• Retrieve with r.as<std::vector<double>>(name).

```
cli.add_double_list("weight");
```

`Flag& Parser::add_positional(std::string name, std::string help = {})`
• Defines a positional argument filled in CLI order; required by default.
• Values are stored as strings; as<int>() et al. convert on read.

```
cli.add_positional("input", "Input file");
```

`Flag& Parser::add_positional_rest(std::string name, std::string help = {})`
• Defines a positional that collects every remaining argument (add at most one, last).
• Never required; an empty remainder reads as an empty vector.

```
cli.add_positional_rest("files", "Everything else");
```

Parser configuration (each returns Parser&)

`Parser& Parser::version(std::string text)`
• Sets a version string and registers the built-in --version/-V switch.
• After a match, r.version_requested() is true.

```
cli.version("myapp 1.0.0");
```

`Parser& Parser::description(std::string text)`
• Overrides/sets the text shown at the top of help().
• Also settable via the constructor.

```
cli.description("Converts images.");
```

`Parser& Parser::epilog(std::string text)`
• Sets text appended verbatim at the bottom of help().
• Use for examples, notes, or exit-code documentation.

```
cli.epilog("Exit codes: 0 ok, 2 bad usage.");
```

`Parser& Parser::allow_unknown(bool on = true)`
• Capture unrecognized flags and surplus positionals instead of failing.
• Retrieve them afterwards via r.unknown().

```
cli.allow_unknown(true);
```

Introspection & output

`const std::string& Parser::program_name() const noexcept`
• Returns the program name used in usage/help output (basename form).
• Handy when building custom error messages.

```
std::cout << cli.program_name();
```

`const std::string& Parser::version_text() const noexcept`
• Returns the version string previously set with version().
• Empty string when no version was registered.

```
std::cout << cli.version_text();
```

`std::vector<const Flag*> Parser::flags() const`
• Returns every defined flag (including hidden ones and built-ins) in definition order.
• Use to dump or audit the current flag set.

```
for (const flageasy::Flag* f : cli.flags()) { /* f->name() */ }
```

`std::string Parser::usage() const`
• Builds the one-line usage string, e.g. 'Usage: myapp -o FILE [options] <input>'.
• Worth printing after an error.

```
std::cerr << cli.usage() << "\n";
```

`std::string Parser::help() const`
• Builds the full help text: description, usage, positionals, options, epilog.
• Honours $COLUMNS for word wrapping; aligned columns.

```
if (r.help_requested()) { std::cout << cli.help(); return 0; }
```

Parsing

`ParseResult Parser::parse(int argc, const char* const argv[]) const`
• Parses argv[1..] (argv[0] is treated as the program name and skipped).
• Never throws for bad user input — all problems are reported through the result.

```
flageasy::ParseResult r = cli.parse(argc, argv);
```

`ParseResult Parser::parse(const std::vector<std::string>& args) const`
• Same, but takes an argument list that does NOT include the program name.
• Ideal for tests and for parsing config/REPL input.

```
auto r = cli.parse(std::vector<std::string>{"-c", "3"});
```

parse() is const and thread-safe on a shared Parser; each call returns an independent, copyable result.

2 · Flag — configure one flag fluently

Obtained from every `add_*` call; each configurator returns `Flag&` so calls chain.

Introspection

`const std::string& name() const noexcept · char short_name() const noexcept`
• name() returns the long name; short_name() returns ' by 0' when none was set.
• Useful when iterating cli.flags().

```
if (f->short_name() == 'v') /* verbose */
```

`ValueType type() const noexcept`
• Returns the value kind: Switch, Integer, Double, or String.
• Match on it when writing generic flag-handling code.

```
if (f->type() == flageasy::ValueType::Integer) /* ... */
```

`bool is_list() / is_positional() / is_rest() / is_required() / is_hidden() / takes_value() const noexcept`
• A family of quick checks: repeatable? positional? rest-collector? required? hidden from help?
• takes_value() is false only for switches.

```
bool needs_value = f->takes_value();
```

`const std::string& help_text() const noexcept`
• Returns the flag's help text shown in help output.
• Empty string if none was given.

```
std::cout << f->help_text();
```

Fluent configurators

`Flag& Flag::help(std::string text)`
• Sets/overrides the help text shown in help output.
• Alternative to passing help to add_*.

```
.help("Overrides the help text")
```

Flag& Flag::metavar(std::string text)
• Sets the value placeholder shown in help (default derives from the name).
• Example: 'FMT' renders as --format FMT.

```
.metavar("FMT")
```

Flag& Flag::required(bool on = true)
• Marks the flag mandatory; parse fails with MissingRequired when absent.
• On positionals it is on by default — use .required(false) to make one optional.

```
.required()
```

Flag& Flag::hidden(bool on = true)
• Omits the flag from generated usage/help output.
• The flag still parses normally.

```
.hidden()
```

Flag& Flag::default_value(bool|const char*|std::string|T value)
• Sets the value used when the flag is not given; still validated at parse time.
• Integral/floating overloads accept any arithmetic type. A bad default is caught immediately.

```
.default_value(1) .default_value("png")
```

Flag& Flag::default_values(std::vector<std::string> values)
• Sets defaults for a list flag (several values at once).
• Also validated.

```
.default_values({"a", "b"})
```

Flag& Flag::range(double low, double high)
• Restricts numeric values to [low, high] inclusive; outside values fail with InvalidValue.
• Applies to int/double flags and their defaults.

```
.range(1, 100)
```

Flag& Flag::choices(std::vector<std::string> allowed)
• Whitelists the raw token; anything else fails with InvalidValue.
• Great for enum-like options (and typo suggestions come free).

```
.choices({"png", "jpeg", "webp"})
```

Flag& Flag::validate(std::function<bool(const std::string&, std::string&)> fn)
• Installs a custom check over the raw token: return false and fill err to reject.
• Runs after type/range/choices checks.

```
.validate([](const std::string& v, std::string& err) { if (v == "raw") { err = "raw is not
supported"; return false; } return true; })
```

3 · ParseResult — read what the user gave you

Status & errors

bool ok() const noexcept · explicit operator bool() const noexcept
• True when the parse succeeded; lets you write if (r) { ... }.
• First thing to check after parse().

```
if (r) { /* use values */ } else { /* report */ }
```

const ParseError& error() const noexcept
• Returns the structured failure: code (ErrorCode), message, flag_name, token.
• Empty (ErrorCode::None) after a successful parse.

```
flageasy::ErrorCode c = r.error().code;
```

const std::string& error_message() const noexcept
• One-line human-readable error, including typo suggestions.
• Empty after a successful parse.

```
std::cerr << r.error_message() << "\n";
```

`bool help_requested() const noexcept`
• True when the user passed the built-in --help/-h.
• Print cli.help() and exit cleanly.

```
if (r.help_requested()) { std::cout << cli.help(); }
```

`bool version_requested() const noexcept`
• True when the user passed --version/-V (needs cli.version(...) set).
• Print your version string and exit.

```
if (r.version_requested()) { std::cout << cli.version_text(); }
```

Raw access

`bool has(const std::string& name) const noexcept`
• Is a flag (or positional) with this long name defined?
• Guards against typos before deeper queries.

```
if (r.has("count")) /* ... */
```

`bool is_set(const std::string& name) const noexcept`
• Was the flag explicitly provided on the command line?
• Lets you distinguish user-supplied values from defaults.

```
if (r.is_set("verbose")) /* user passed -v */
```

`const std::vector<std::string>& raw(const std::string& name) const`
• Returns the effective raw values with defaults applied.
• Throws std::out_of_range for an unknown name.

```
auto vals = r.raw("tag");
```

`std::optional<std::string> value(const std::string& name) const`
• Returns the first effective raw value, or nullopt when there is none.
• Convenience over raw()[0].

```
auto v = r.value("name");
```

Typed access

`template<class T> T as(const std::string& name) const`
• Converts to T: bool, std::string, arithmetic types, or std::vector of those.
• Throws std::runtime_error when unknown/missing/multi-value/unconvertible.

```
int c = r.as<int>("count");
```

`template<class T> std::optional<T> try_as(const std::string& name) const`
• Like as<T> but returns nullopt instead of throwing.
• Best for optional or possibly-invalid input.

```
auto d = r.try_as<double>("scale");
```

`template<class T> T as_or(const std::string& name, T fallback) const`
• Like as<T> but silently falls back to the given value on any failure.
• One-liner for 'value or my default'.

```
int c = r.as_or<int>("count", 5);
```

`auto v = r.as<std::vector<std::string>>("tag"); // list flags`
• Vector retrieval gathers every occurrence of a list flag in CLI order.
• Scalar as<T> on a multi-value flag throws — use the vector form.

```
auto tags = r.as<std::vector<std::string>>("tag");
```

Enumeration & reporting

`const std::vector<ParsedFlag>& flags() const noexcept`
• Every defined flag/positional in definition order with effective values and is_set markers.
• Includes the built-in help/version flags — great for dumping/debugging.

```
for (const auto& pf : r.flags()) std::cout << pf.name << "=" << pf.value();
```

`const std::vector<std::string>& positionals() const noexcept`
• Positional values in the order they appeared on the command line.
• Doubles as a quick lookup for rest-collectors.

```
for (const auto& p : r.positionals()) /* ... */
```

`const std::vector<std::string>& unknown() const noexcept`
• Arguments captured because allow_unknown(true) was set.
• Empty unless unknown-capture is enabled.

```
for (const auto& u : r.unknown()) /* ... */
```

`std::vector<std::string> names() const`
• Returns the names of all defined flags/positionals.
• Handy for validation loops or diagnostics.

```
for (const auto& n : r.names()) /* ... */
```

`std::string to_string() const`
• One-line summary like 'count=3, verbose=true, name=<unset>'.
• Perfect for logs and --dry-run style debugging.

```
std::cout << r.to_string() << "\n";
```

ParsedFlag fields

Each `ParsedFlag` exposes: `name` (long name), `short_name`, `type`, `is_positional`, `is_set`, `values` (effective), and `value()` — the first value or "" when there is none.

4 · Free functions & error codes

`namespace flageasy { const char* library_version() noexcept; / _major / _minor / _patch }`
• Reports the library version as a string and as int components.
• Use for runtime capability checks or logs.

```
std::cout << flageasy::library_version(); // "1.0.0"
```

`std::string_view to_string(ValueType) · std::string_view to_string(ErrorCode)`
• Converts the enums to stable textual names ('Switch', 'UnknownFlag', ...).
• Use when printing diagnostics.

```
std::cout << flageasy::to_string(r.error().code);
```

ErrorCode	Meaning
`None`	no error — parse succeeded
`UnknownFlag`	unrecognized --flag or -f
`MissingValue`	an option required a value that was not given
`InvalidValue`	value failed type / range / choices / validator
`MissingRequired`	a required flag or positional was not provided
`UnexpectedArgument`	extra positional with no slot for it

Standard failure-handling recipe

```
if (r.help_requested()) { std::cout << cli.help(); return 0; } if (r.version_requested()) {
std::cout << cli.version_text(); return 0; } if (!r) { std::cerr << "error: " <<
r.error_message() << "\n\n" << cli.usage() << "\n"; return 2; }
```

FlagEasy v1.0.0 · header: flageasy/flageasy.hpp · build: cmake --install build --prefix <prefix>, then find_package(flageasy 1.0 REQUIRED) and link flageasy::flageasy. Only definition-time misuse throws (std::invalid_argument); bad user input never throws.